



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SE/SS ZG566

Secure Software Engineering

T V Rao



- *The slides presented here are obtained from the authors of the books, product documentations, and from various other contributors. I hereby acknowledge all the contributors for their material and inputs.*
- *I have added and modified slides to suit the requirements of the course.*

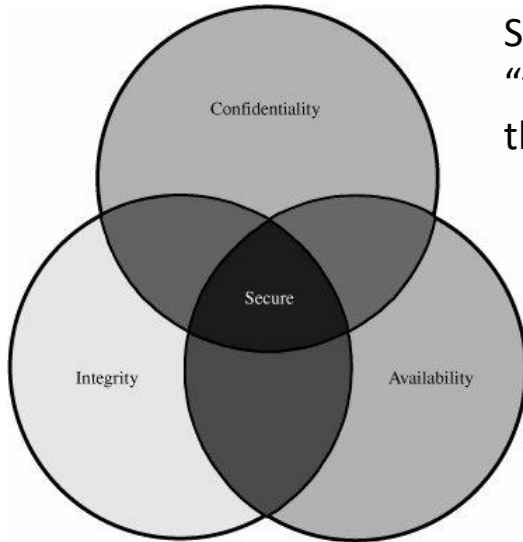


BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Mid-semester Review

Security Principles



Saltzer and Schroeder defined security as “techniques that control who may use or modify the computer or the information contained in it”

Integrity

Information be protected from improper modification

Availability

Available to user or program with legitimate right

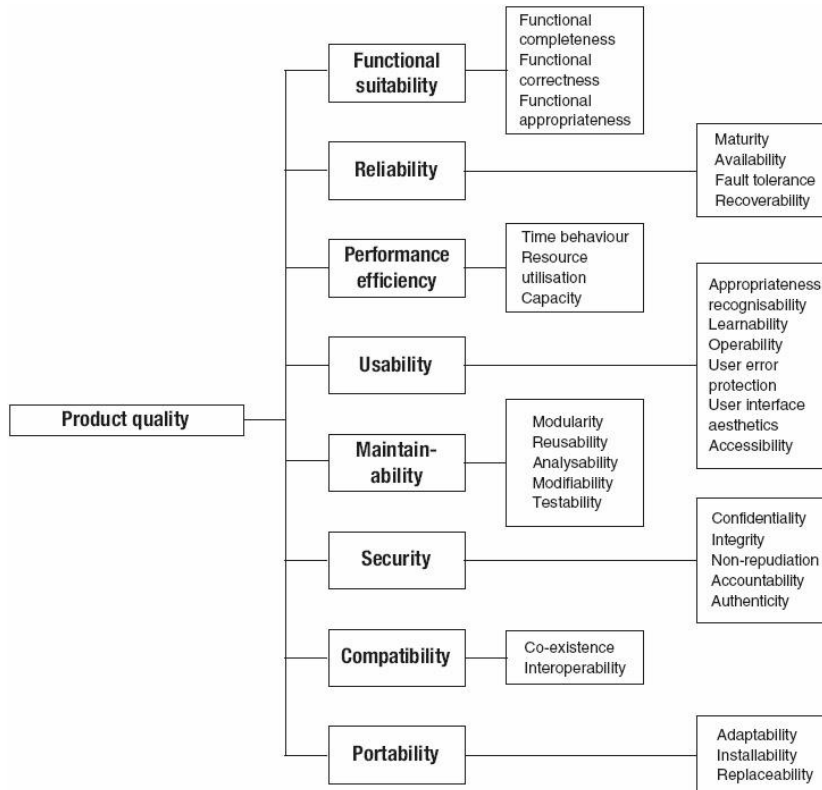
Confidentiality

Protection of data from unauthorized disclosure

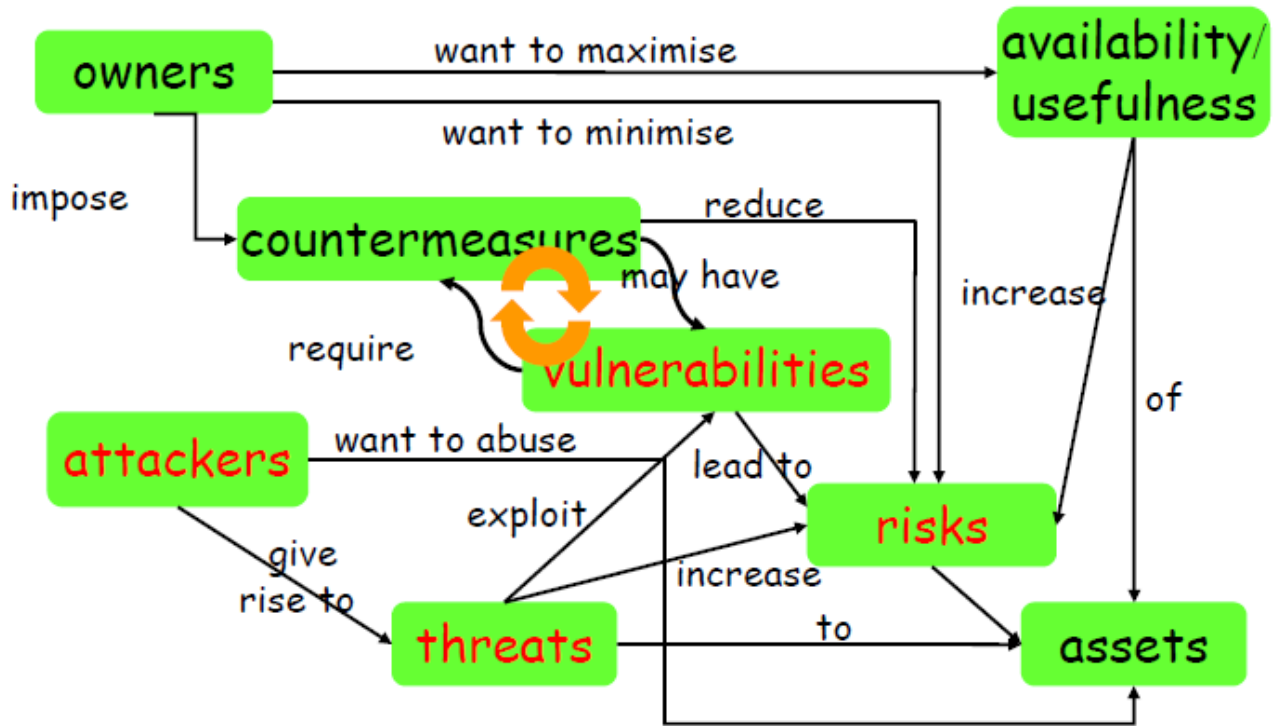
Two additional properties commonly associated with human users are required in software entities that act as users, e.g. proxy agents, web services etc.

- Accountability : All security-relevant actions of the software-as-user must be recorded and tracked with attribution, both while and after the recorded action occurs
- Non-repudiation : Ability to prevent the software-as-user from disproving or denying responsibilities for actions it has performed

Product quality model of ISO/IEC 25010



Security Concepts and Relationships



Security Concepts and Relationships

The **attack surface** of a software environment is the sum of the different points (the "attack vectors") where an unauthorized user (the "attacker") can try to enter data to or extract data from an environment.

An **attack vector** is a path or means by which a hacker (or cracker) can gain access to a computer or network server in order to deliver a payload or malicious outcome.

If an attack vector is thought of as a guided missile (e.g. email), its payload can be compared to the warhead (e.g. malicious attachment) in the tip of the missile.

Social engineering attack is based on deceiving end users or administrators at a target site. Such attacks are typically carried out by email or by contacting users by phone and impersonating an authorized user, in an attempt to gain unauthorized access to a system or application

Attack surfaces

Attack surface: the reachable and exploitable vulnerabilities in a system

- Open ports
- Services outside a firewall
- An employee with access to sensitive info
- ...

Three categories

- **Network attack surface** (i.e., network vulnerability)
- **Software attack surface** (i.e., software vulnerabilities)
- **Human attack surface** (e.g., social engineering)

Attack analysis: assessing the scale and severity of threats

The Asymmetric Problem of Security

- No matter how much effort we expend, we will never get code 100 percent correct
 - This is an asymmetric problem
 - **We** must be 100 percent correct, 100 percent of the time, on a schedule, with limited resources, only knowing what we know today
 - Oh, and the product has to be reliable, supportable, compatible, manageable, affordable, accessible, usable, global, doable, deployable...
 - **They** can spend as long as they like to find one bug, with the benefit of future research

The Asymmetric Problem of Security

Tools make it easy to build exploit code

Reverse engineering tools

- Structural Comparison of Executable Objects, Halvar Flake
 - http://www.zynamics.com/downloads/dimva_paper2.pdf
 - PCT Bug: "Detecting and understanding the vulnerability took less than 30 minutes"
 - H.323 ASN.1 Bug: "The total analysis took less than three hours time"

Exploit payloads

- www.metasploit.com



CVEs & CWEs

NIST maintains CVE Database.

(Other agencies/companies also do aggregation with traceability)

The CVEs are mapped to CWEs maintained by MITRE.



Structured Approach to Threat Modeling

According to Adam Shostack, you begin threat modeling by focusing on four key questions

What are you building?

What can go wrong?

What should you do about those things that can go wrong?

Did you do a decent job of analysis (retrospect)



Structured Approach to Threat Modeling

People often use an approach centered on

- Models of their assets (Valuable things they have),
- Models of attackers (People who might go after assets), or
- Models of their software (Common way to attack is via the deployed software)

Centering on one of these is preferable to using approaches that combine them because the combinations tend to be confusing

According to Adam Shostack, first two sets of models help in engaging with non-technical people and third type of models are important

Threat modeling process for Software

According to OWASP, the threat modeling process for software application can be decomposed into 3 high level steps

- Decompose the Application
- Determine and rank threats
- Determine countermeasures and mitigation

Decompose the Application

Understanding of the application and how it interacts with external entities.

- involves creating use-cases to understand how the application is used,
- identifying entry points to see where a potential attacker could interact with the application,
- identifying assets i.e. items/areas that the attacker would be interested in, and
- identifying trust levels which represent the access rights that the application will grant to external entities.

Produce data flow diagrams (DFDs) for the application. The DFDs show the different paths through the system, highlighting the privilege boundaries.

Determine and rank threats

A threat categorization such as STRIDE can be used, or the Application Security Frame (ASF) that defines threat categories such as Auditing & Logging, Authentication, Authorization, Configuration Management, Data Protection in Storage and Transit, Data Validation, Exception Management.

The threat categorization helps to identify threats both from the attacker (STRIDE) and the defensive perspective (ASF).

DFDs produced in step 1 help to identify the potential threat targets from the attacker's perspective, such as data sources, processes, data flows, and interactions with users.

These threats can be identified further as the roots for threat trees; there is one tree for each threat goal. From the defensive perspective, ASF categorization helps to identify the threats as weaknesses of security controls for such threats.

Use and abuse cases can illustrate how existing protective measures could be bypassed, or where a lack of such protection exists.



Determine countermeasures and mitigation

A lack of protection against a threat might indicate a vulnerability whose risk exposure could be mitigated with the implementation of a countermeasure.

Countermeasures can be identified using threat-countermeasure mapping lists.

The determination of the security risk for each threat can be determined using a value-based risk model such as DREAD

Based on risk ranking assigned to the threats, it is possible to sort threats from the highest to the lowest risk, and prioritize the mitigation effort, such as by responding to such threats by applying the identified countermeasures

Identify Threats

Experts can brainstorm

How to do this without being an expert?

- Use STRIDE to step through the diagram elements
- Get specific about threat manifestation

Threat

Spoofing

Tampering

Repudiation

Information Disclosure

Denial of Service

Elevation of Privilege

Property we want

Authentication

Integrity

Nonrepudiation

Confidentiality

Availability

Authorization

STRIDE Threat & Mitigation Techniques List



Threat Type	Mitigation Techniques
Spoofing Identity	1.Appropriate authentication 2.Protect secret data 3.Don't store secrets
Tampering with data	1.Appropriate authorization 2.Hashtes 3.MACs 4.Digital signatures 5.Tamper resistant protocols
Repudiation	1.Digital signatures 2.Timestamps 3.Audit trails
Information Disclosure	1.Authorization 2.Privacy-enhanced protocols 3.Encryption 4.Protect secrets 5.Don't store secrets
Denial of Service	1.Appropriate authentication 2.Appropriate authorization 3.Filtering 4.Throttling 5.Quality of service
Elevation of privilege	1.Run with least privilege

DREAD

In the Microsoft DREAD threat-risk ranking model, the technical risk factors for impact are Damage and Affected Users, while the ease of exploitation factors are Reproducibility, Exploitability and Discoverability. This risk factorization allows the assignment of values to the different influencing factors of a threat. To determine the ranking of a threat, the threat analyst has to answer basic questions for each factor of risk

- For Damage: How big would the damage be if the attack succeeded?
- For Reproducibility: How easy is it to reproduce an attack to work?
- For Exploitability: How much time, effort, and expertise is needed to exploit the threat?
- For Affected Users: If a threat were exploited, what percentage of users would be affected?
- For Discoverability: How easy is it for an attacker to discover this threat?

SQUARE Process Model

- Security Quality Requirements Engineering (SQUARE) is a process model that was developed at Carnegie Mellon University [Mead 2005].
- SQUARE provides a means for eliciting, categorizing, and prioritizing security requirements for information technology systems and applications.
- The focus of the model is to build security concepts into the early stages of the SDLC. It can also be used for documenting and analyzing the security aspects of systems once they are implemented in the field and for steering future improvements and modifications to those systems.
- The SQUARE work is supported by the Army Research Office through grant (“Perpetually Available and Secure Information Systems”) to Carnegie Mellon University’s CyLab.

SQUARE Process Steps

1. Agree on definitions
2. Identify security goals
3. Develop artifacts to support security requirements definition
4. Perform (security) risk assessment
5. Select elicitation techniques
6. Elicit security requirements
7. Categorize requirements as to level (e.g., system, software) and whether they are requirements or other kinds of constraints
8. Prioritize requirements
9. Inspect requirements

Step 3: Develop Artifacts

- Architectural diagrams, use cases, misuse cases, abuse case diagrams, attack trees, and essential assets and services were documented in this step.
- Attack patterns provide a good starting point for developing artifacts that reflect the attacker's perspective

Attack Patterns

Attack patterns are descriptions of common methods for exploiting software. Act as a mechanism to capture and communicate the attacker's perspective.

They derive from the concept of design patterns [Gamma 95] applied in a destructive rather than constructive context and are generated from in-depth analysis of specific real-world exploit examples

The following typical information is captured for each attack pattern:

- Pattern name and classification
- Attack prerequisites
- Description
- Targeted vulnerabilities or weaknesses
- Method of attack
- Attacker goal
- Attacker skill level required
- Resources required
- Blocking solutions
- Context description
- References

Attack Patterns

Pattern Name and Classification: A unique, descriptive identifier for the pattern.

Attack Prerequisites: What conditions must exist or what functionality and what characteristics must the target software have, or what behavior must it exhibit, for this attack to succeed?

Description: A description of the attack including the chain of actions taken.

Related Vulnerabilities or Weaknesses: What specific vulnerabilities or weaknesses does this attack leverage?

Method of Attack: What is the vector of attack used (e.g., malicious data entry, maliciously crafted file, protocol corruption etc.)?

Attack Motivation-Consequences: What is the attacker trying to achieve by using this attack?

Attacker Skill or Knowledge Required: What level of skill or specific knowledge must the attacker have to execute such an attack?

Resources Required: What resources (e.g., CPU cycles, IP addresses, tools, time) are required to execute the attack?

Solutions and Mitigations: What actions or approaches are recommended to mitigate this attack, either through resistance or through resiliency?

Context Description: In what technical contexts (e.g., platform, OS, language, architectural paradigm) is this pattern relevant? This information is useful for selecting a set of attack patterns that are appropriate for a given context.

References: What further sources of information are available to describe this attack?

Attack Patterns

Pattern name and classification: Shell Command Injection—Command Delimiters

Attack Prerequisites: The application must pass user input directly into a shell command.

Description: Using the semicolon or other off-nominal characters, multiple commands can be strung together. Unsuspecting target programs will execute all the commands. An example may be when authenticating a user using a web form, where the username is passed directly to the shell as in: `exec("cat data_log_" + userInput + ".dat")`

The "+" sign denotes concatenation. The developer expects that the user will only provide a username. However, a malicious user could supply `"username.dat; rm -rf / ;"` as the input to execute the malicious commands on the machine running the target software. In the above case, the actual commands passed to the shell will be: `cat data_log_username.dat; rm -rf / ;.dat`

The first command may or may not succeed; the second command will delete everything on the file system to which the application has access, and success/failure of the last command is irrelevant.

Related Vulnerabilities or Weaknesses: CWE-OS Command Injection, CVE-1999-0043, CVE-1999-0067, CVE-1999-0097, CVE-1999-0152, CVE-1999-0210, CVE-1999-0260, 1999-0262, CVE-1999-0279, CVE-1999-0365, etc.

Method of Attack: By injecting other shell commands into other data that are passed directly into a shell command.

Attack Motivation-Consequences: Execution of arbitrary code.

Attacker Skill or Knowledge Required: Finding and exploiting this vulnerability does not require much skill.

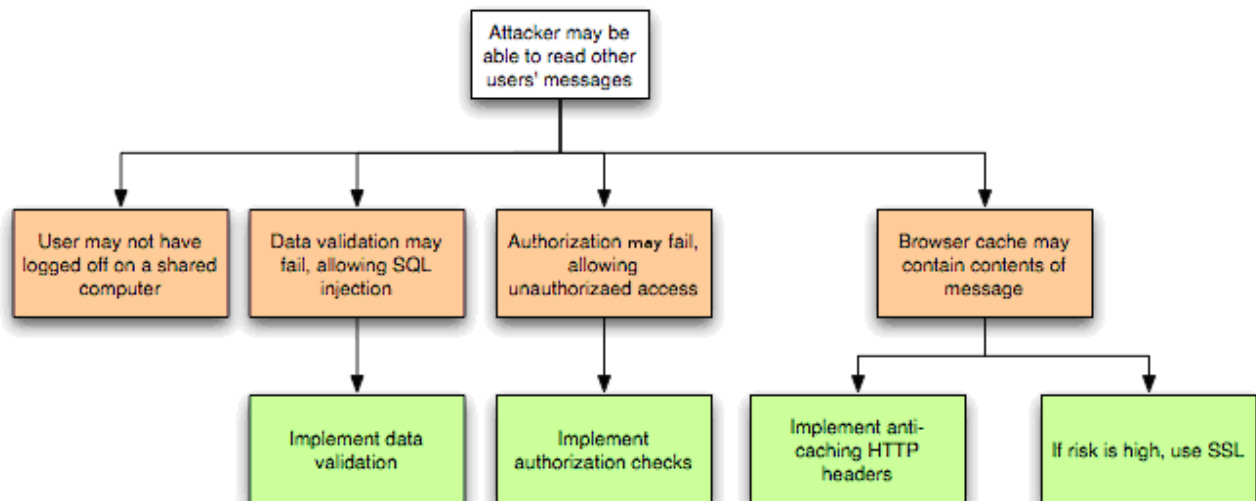
Resources Required: No special or extensive resources are required for this attack.

Solutions and Mitigations: Define valid inputs to all fields and ensure that the user input is always valid. Also perform white-list and/or black-list filtering as a backup to filter out known command delimiters.

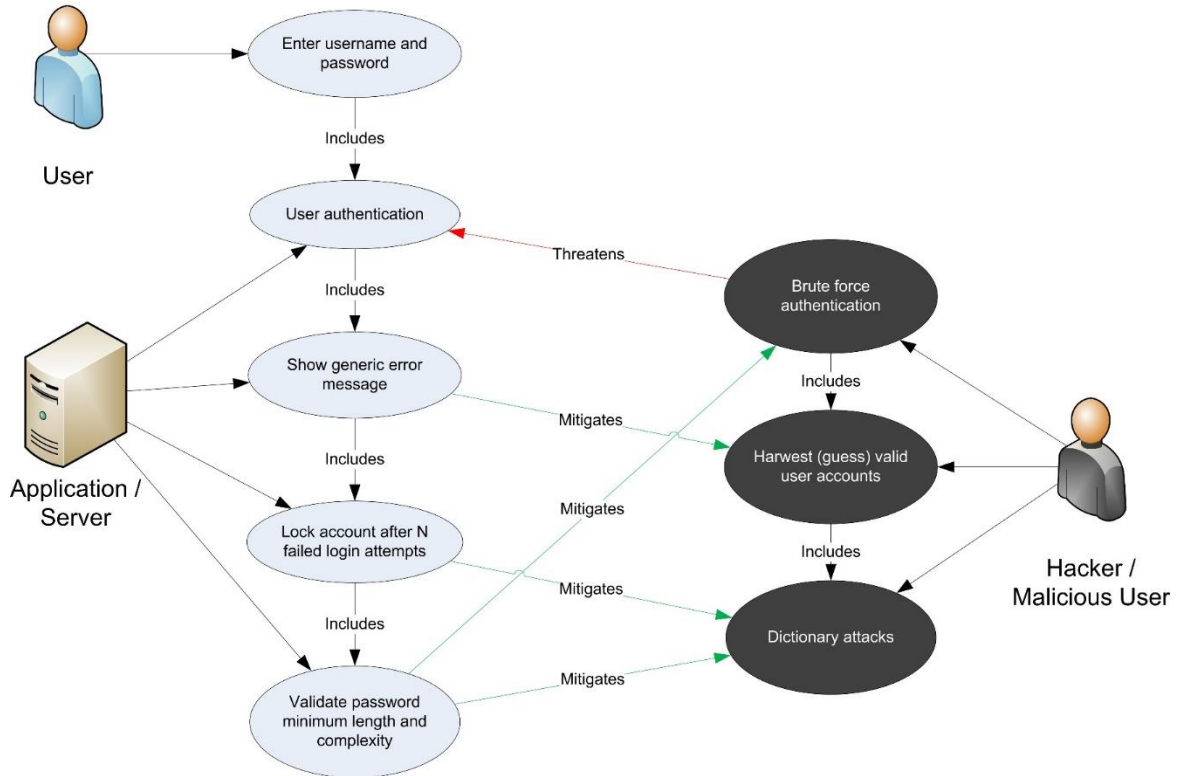
Context Description: OS: UNIX.

References: Exploiting Software [Hoglund 04].

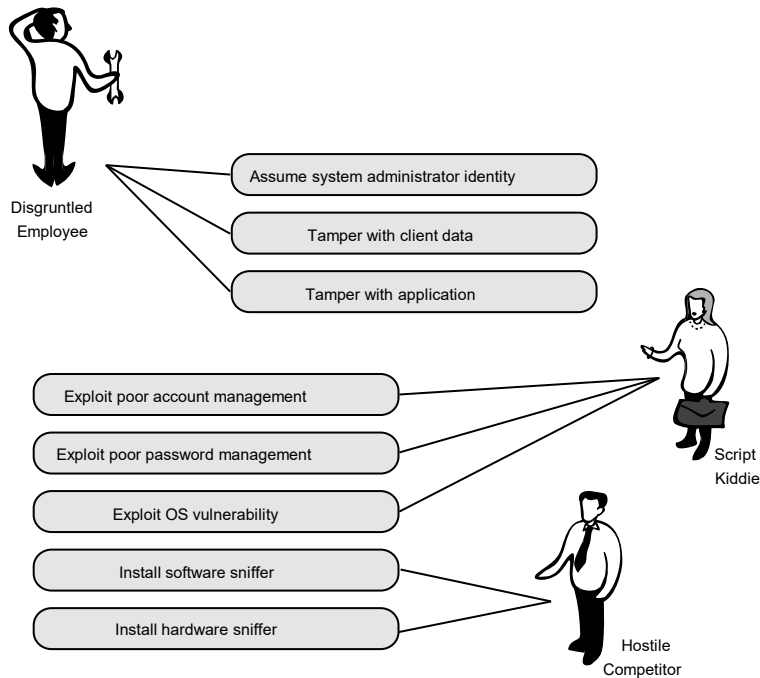
A Threat Tree



Use Case with Abuse Case



Abuse case example



SAMM - Consider security explicitly – SR1

(during the software requirements process)

Goal	Activities	Quality Criteria
Consider security explicitly during the software requirements process.	Identify security requirements	- Teams derive security requirements from functional requirements and customer or organization concerns - Security requirements are specific, measurable, and reasonable - Security requirements are in line with the organizational baseline
	Perform vendor assessments	- Consider including specific security requirements, activities, and processes when creating third-party agreements - A vendor questionnaire is available and used to assess the strengths and weaknesses of your suppliers

SDL : Establish Security & Privacy Requirements

Define and integrate security and privacy requirements early

- identify key milestones and deliverables and minimize disruptions to plans and schedules.

Security and privacy analysis including

- assigning security experts,
- defining minimum security and privacy criteria for an application, and
- deploying a security vulnerability/work item tracking system.

When should this practice be implemented?

- Traditional Software development: Requirements Phase
- Agile development: One Time

SDL : Create Quality Gates/Bug Bars

Defining minimum acceptable levels of security and privacy quality

- the team understand risks associated with security issues,
- Team identifies and fixes security bugs during development, and
- Team apply the standards throughout the entire project.

Set a bug bar to clearly define the severity thresholds of security vulnerabilities

- (for example, no known vulnerabilities in the application with a “critical” or “important” rating at time of release)

When should this practice be implemented?

- Traditional Software development: Requirements Phase
- Agile development: One Time

Architectural Risk Analysis

- The risk assessment methodology encompasses six fundamental activity stages:
 - software characterization
 - architectural vulnerability assessment
 - threat analysis
 - risk likelihood determination
 - risk impact determination
 - risk mitigation

Design Principles for Software Security

- Securing the Weakest Link
- Defense in Depth
- Failing Securely
- Least Privilege
- Separation of Privilege
- Economy of Mechanism
- Least Common Mechanism
- Reluctance to Trust
- Never Assuming that your Secrets are Safe
- Complete Mediation
- Psychological Acceptability
- Promoting Privacy

Classes of Secure Patterns

Architectural-level patterns. Architectural-level patterns focus on the high-level allocation of responsibilities between different components of the system and define the interaction between those high-level components.

- Distrustful Decomposition
- PrivSep (Privilege Separation)
- Defer to Kernel

Design-level patterns. Design-level patterns describe how to design and implement pieces of a high-level system component, that is, they address problems in the internal design of a single high-level component, not the definition and interaction of high-level components themselves.

- Secure Factory
- Secure Strategy Factory
- Secure Builder Factory
- Secure Chain of Responsibility
- Secure State Machine
- Secure Visitor

Implementation-level patterns. Implementation-level patterns address low-level security issues. Patterns in this class are usually applicable to the implementation of specific functions or methods in the system. Implementation-level patterns address the same problem set addressed by the CERT Secure Coding Standards

- Secure Logger
- Clear Sensitive Information
- Secure Directory
- Input Validation

Software Testing Key Issues (SWEBOK)

Dynamic: The input value alone is not always sufficient to specify a test, since a complex, nondeterministic system might react to the same input with different behaviors, depending on the system state.

Finite: Even in simple programs, so many test cases are theoretically possible that exhaustive testing could require months or years to execute.

Selected: How to identify the most suitable test set under given conditions is a complex problem; in practice, risk analysis techniques and software engineering expertise are applied.

Expected: It must be possible, although not always easy, to decide whether the observed outcomes of program testing are acceptable or not; otherwise, the testing effort is useless.

Security Testing (SWEBOK)

- Security testing is focused on the verification that the software is protected from external attacks.
- Security testing verifies the confidentiality, integrity, and availability of the systems and its data.
- Security testing includes verification against misuse and abuse of the software or system (negative testing).

Penetrate and Patch

A number of software tools exist to support *post facto* security analysis of installed computer systems. Examples include

- Nmap (Network Mapper) a free and open-source network scanner.
- Burp Intruder is a powerful tool for automating customized attacks against web applications.
- Metasploit scans vulnerabilities backed by open-source database of known exploits.

Such tools are reactive.

- Security analysis needs to be performed as part of the software development process, before software is released.
- Crackers often know about vulnerabilities before system administrators
- System administrators have neither the time nor the inclination to patch software if they have not noticed any security breaches.
- While a patch may close one security hole, it may simultaneously open up others.

Source Code Analysis Tools

Source Code Analysis Tools (security analyzers) are automated tools for helping analysts find security-related problems in software

- use data- and control-flow analysis to find subtler bugs and to reduce false alarms

Some vulnerabilities (e.g. use of `strcpy()`) can be detected with high accuracy, others are harder to detect, and, in fact, one can always devise vulnerabilities that are undetectable altogether.

Tools have tradeoff between false alarms (also known as false positives) and missed vulnerabilities (also known as false negatives)

- can be configured to make a tool more sensitive (decreasing false negatives while increasing false positives) or make it less sensitive (increasing false negatives while decreasing false positives)

Some of the tools are listed at

<https://resources.infosecinstitute.com/topic/secure-coding-top-15-code-analysis-tools/>

Capabilities of Security Analyzers

- Examining Calls to Potentially Insecure Library Functions
- Detecting Bounds-Checking Errors and Scalar Type Confusion
- Detecting Type Confusion Among References or Pointers
- Detecting Memory Allocation Errors
- Detecting Vulnerabilities that Involve Sequences of Operations (Control-Flow Analysis)
- Data-Flow Analysis
- Pointer-Aliasing Analysis
- ...

White Box Testing Techniques

Data-Flow Analysis

Code-Based Fault Injection

Abuse Cases

Trust Boundaries Mapping

Code Coverage Analysis

Black Box Testing

Black box tests can help

- identify implementation errors that were not discovered during code reviews, unit tests, or security white box tests

- discover potential security issues resulting from boundary conditions that were difficult to identify and understand during the design and implementation phases

- uncover security issues resulting from incorrect product builds (e.g., old or missing modules/files)

- detect security issues that arise as a result of interaction with underlying environment (e.g., improper configuration files, unhardened OS and applications)

Fuzz Testing - Kinds of fuzzing

- **Black box**
 - The tool knows nothing about the program or even its inputs.
Limited benefits
- **Grammar based**
 - The tool generates input based on known grammar
- **White box**
 - The tool generates new inputs based on the code of the program.
Computationally complex

Penetration Testing - Skills

- A pen tester needs to know a lot about the target domain.
- For example, if the pen tester is attacking web applications, then the pen tester needs to know how the web works. Need to know how systems are built in that domain.
 - What protocols allow applications to communicate. For the web, that's HTTP and TCP, and IP.
 - The languages that are used to build applications like PHP, Java, or Ruby for talking about the web.
 - Frameworks that used to build applications or application components like, for the web, Ruby on Rails, DreamWeaver, Drupal, and so on.
- Pen tester also need to know common weaknesses from that domain.
 - For example, the bugs that are common to web applications like SQL injections or cross-site scripting, or cross-site request forgery. Or common misconfigurations or bad designs, like the use of default passwords or hidden files.

Categories of Penetration Testing Tools

- **Host-Based Tools**
- **Network-Based Tools**
- **Application Testing Proxies**
- **Application Scanning Tools**
- Tools integrated with other IT security technologies, viz. firewalls, intrusion detection and prevention systems



Computer Security: Principles and Practice by William Stallings, and Lawrie Brown Pearson, 2018.

Software Security Engineering, Julia H. Allen, et al, Pearson, 2008.

www.microsoft.com

www.owasp.com

sei.cmu.edu



Thank You!